DOCKER: FUNDAMENTOS Y RELEVANCIA EN LA ARQUITECTURA DE SOFTWARE

Informe de investigación · Parte 1

Estudiante: ADRIAN MONTES STAVE

Código: T00081565

Asignatura: Arquitectura de Software

Actividad: Taller del 2.º corte — Semana 10

2026

Introducción

En el desarrollo de software no basta con que una aplicación funcione en el computador de quien la programó; también debe poder ejecutarse de forma confiable en los equipos del resto del grupo, en pruebas y en producción. Docker ayuda a resolver este reto al empaquetar una aplicación con los componentes y las dependencias que necesita para ejecutarse. En este informe se explican sus conceptos principales, componentes, funcionalidades y comandos, y se relaciona su uso con estilos arquitectónicos como microservicios y arquitectura orientada a eventos.

1. ¿Qué es Docker?

Docker es una plataforma para construir, distribuir y ejecutar aplicaciones dentro de contenedores. Un contenedor reúne la aplicación y sus dependencias —por ejemplo, bibliotecas, archivos y configuración— en un entorno aislado. De esta manera, el equipo puede transportar una imagen entre máquinas que tengan un motor de contenedores compatible, reduciendo las diferencias entre desarrollo, pruebas y producción.

A alto nivel, el motor de Docker crea y administra contenedores a partir de imágenes. Los contenedores comparten el kernel del sistema operativo anfitrión (host), pero mantienen aislamiento de procesos, red y sistema de archivos según su configuración. Por eso normalmente requieren menos recursos y arrancan más rápido que una máquina virtual tradicional, que incluye un sistema operativo invitado completo sobre un hipervisor. Los contenedores no son máquinas virtuales: el aislamiento es distinto y no elimina la necesidad de aplicar medidas de seguridad.

2. Componentes principales

CLI (cliente docker): es la herramienta de terminal desde la que el usuario escribe órdenes como `docker build`, `docker pull` o `docker ps`. **Daemon (dockerd):** servicio que recibe solicitudes y administra imágenes, contenedores, redes y volúmenes. **Imagen:** plantilla de solo lectura con las capas y archivos necesarios para crear contenedores. **Contenedor:** instancia ejecutable creada a partir de una imagen. **Dockerfile:** archivo de instrucciones que describe cómo construir una imagen. **Registro:** servicio que almacena y distribuye imágenes; Docker Hub es un ejemplo.

Cadena de construcción: el desarrollador escribe un Dockerfile y ejecuta `docker build`; Docker produce una imagen. Después, `docker run` crea y arranca un contenedor basado en ella. El registro interviene cuando se descargan imágenes con `docker pull` o se publican con `docker push`. Una imagen también puede construirse localmente sin publicarse.

```
Esquema: Dockerfile → docker build → Imagen → docker run → Contenedor. Registro ↔ descarga/publicación de imágenes.
```

3. Funcionalidades principales

Portabilidad. La imagen puede usarse en distintos equipos con Docker y una arquitectura compatible. Ejemplo: una API empaquetada por el equipo puede probarse en el portátil de otro integrante sin instalar manualmente todas las dependencias.

Aislamiento. Los servicios se ejecutan separados, lo que reduce conflictos de bibliotecas y procesos. Ejemplo: MySQL y una API pueden estar en contenedores distintos, con redes y variables configuradas para comunicarse.

Reproducibilidad. El Dockerfile y la configuración versionada describen cómo preparar el entorno. Ejemplo: cada integrante construye la misma imagen de la API con una versión acordada de Go.

Versionado de imágenes. Las etiquetas permiten identificar versiones, por ejemplo `mysql:8`. Conviene fijar versiones adecuadas y no depender ciegamente de etiquetas cambiantes como `latest`.

Escalabilidad. Se pueden ejecutar varias instancias de un servicio, siempre que la aplicación, la red y la infraestructura estén preparadas. Ejemplo: aumentar las réplicas de una API para atender más solicitudes.

Automatización y CI/CD. Las herramientas de integración continua pueden construir, probar y publicar imágenes de manera repetible. Ejemplo: después de aprobar pruebas, una canalización crea una imagen versionada para desplegarla.

4. Comandos principales

Comando	¿Qué hace?	Ejemplo
docker pull	Descarga una imagen desde un registro.	docker pull mysql:8
docker run	Crea y ejecuta un contenedor.	docker run --name mi-mysql -e MYSQL_ROOT_PASSWORD=clave -d mysql:8
docker ps	Muestra contenedores en ejecución.	docker ps
docker ps -a	Muestra contenedores activos y detenidos.	docker ps -a
docker logs	Muestra la salida registrada de un contenedor.	docker logs biblioteca-api
docker exec	Ejecuta un comando dentro de un contenedor activo.	docker exec -it biblioteca-api sh
docker stop / start	Detiene o inicia un contenedor existente.	docker stop biblioteca-api
docker rm	Elimina un contenedor detenido.	docker rm biblioteca-api
docker images	Lista imágenes locales.	docker images
docker rmi	Elimina una imagen local no utilizada.	docker rmi mi-api:1.0
docker build	Construye una imagen a partir de un Dockerfile.	docker build -t mi-api:1.0 .
docker compose up	Crea e inicia los servicios definidos en Compose.	docker compose up --build

Tres comandos explicados en detalle

1. docker run. Crea y ejecuta un contenedor a partir de una imagen. `-d` lo deja en segundo plano; `--name` asigna un nombre; `-p 8080:8080` publica un puerto del contenedor en el host; `-e CLAVE=valor` define una variable de entorno; `-v volumen:/ruta` monta un volumen o una ruta persistente. Lo usaría para probar rápidamente un servicio o ejecutar una base de datos de desarrollo. Las contraseñas reales no deben quedar publicadas en comandos o repositorios.

2. docker build. Construye una imagen con las instrucciones de un Dockerfile. La opción `-t mi-api:1.0` etiqueta la imagen con nombre y versión, y el punto final (`.`) indica el contexto de construcción actual. Lo usaría después de modificar el código o las dependencias para generar una versión reproducible de la aplicación.

3. docker compose up. Lee el archivo `compose.yaml` o `docker-compose.yml` y crea/inicia los servicios declarados, junto con sus redes y volúmenes. `--build` solicita construir las imágenes que lo requieran; `-d` deja los servicios en segundo plano. Lo usaría para iniciar una API y su base de datos como una sola aplicación coordinada.

5. ¿Qué problema resuelve Docker?

El problema conocido como “en mi máquina funciona” aparece cuando el programa depende de condiciones que no son iguales en todos los equipos. Una computadora puede tener una versión distinta de una biblioteca, otro sistema operativo, variables de entorno diferentes, puertos ocupados o instalaciones manuales incompletas. Por ejemplo, una API podría funcionar con una versión de Go y fallar en el computador de un compañero porque allí falta una dependencia o se usa una versión incompatible de MySQL.

Antes: cada integrante instala Go, MySQL y las bibliotecas por separado; las configuraciones se hacen manualmente y los errores son difíciles de reproducir. **Después:** el equipo mantiene un Dockerfile, una imagen versionada y un archivo Compose con los servicios y la configuración necesaria. Los integrantes ejecutan los mismos pasos y reducen diferencias. Docker no corrige errores lógicos ni garantiza por sí solo que cualquier configuración sea segura, pero mejora la consistencia del entorno.

6. Importancia en Arquitectura de Software

En una arquitectura distribuida conviene separar servicios por responsabilidad. Una API, una base de datos y un broker de eventos pueden ejecutarse en contenedores distintos; así se configuran, actualizan y supervisan de manera independiente.

Si se cambia la API, por ejemplo, no es necesario reconstruir la imagen de MySQL. La separación también facilita asignar redes, puertos, variables y almacenamiento según las necesidades de cada servicio.

Docker ayuda a pasar un entorno similar de desarrollo a pruebas y producción, lo que disminuye diferencias entre etapas. Además, facilita ejecutar más instancias de un servicio y trasladar imágenes a infraestructura local o en la nube compatible. La escalabilidad real también depende de la capacidad disponible, el balanceo de carga, el estado de la aplicación y la orquestación.

En **microservicios**, los contenedores permiten empaquetar y desplegar cada servicio de manera independiente. En **SOA**, ayudan a distribuir servicios con responsabilidades definidas. En una arquitectura **orientada a eventos**, la API, el broker y los consumidores pueden ejecutarse como servicios separados conectados por una red. Docker no impone una arquitectura concreta, pero ofrece una forma práctica de implementar y operar sus componentes.

Conclusión

Considero que Docker es importante porque convierte la preparación del entorno en un proceso documentado y repetible, en vez de depender de instalaciones manuales en cada computador. En un proyecto de biblioteca, por ejemplo, usaría contenedores para la API y MySQL, con Docker Compose para levantarlos juntos. Esto haría más sencillo que los integrantes prueben los mismos endpoints, identifiquen errores y preparen el proyecto para su despliegue. También tendría cuidado de no guardar secretos en el repositorio y de respaldar los datos que deban conservarse.

Referencias

Docker. (s. f.). *Docker overview*. https://docs.docker.com/get-started/docker-overview/

Docker. (s. f.). *Dockerfile reference*. https://docs.docker.com/reference/dockerfile/

Docker. (s. f.). *Docker Compose overview*. https://docs.docker.com/compose/

Docker. (s. f.). *Docker CLI reference*. https://docs.docker.com/reference/cli/docker/